MATH3881/5881: Statistical Machine Learning Theory

Week 1: Introduction to Machine Learning — Exercises and Solutions

Sarat Moka *UNSW, Sydney*

Exercise 1.1.1 (Learning Paradigms: Face Recognition)

Think about how you learned to recognise faces. Was this closer to supervised learning, unsupervised learning, or a mixture of both?

Solution. Face recognition is best described as a **mixture** of both paradigms. In infancy, faces are encountered without explicit labels, the visual system clusters similar patterns (two eyes, a nose, a mouth) through repeated exposure, which resembles *unsupervised* learning. Over time, adults attach names to faces ("That's Grandma"), introducing labeled supervision. The process is therefore closer to *semi-supervised* or *self-supervised* learning in the cognitive sense.

In modern machine learning, face recognition is implemented as *supervised learning*: a deep neural network is trained on large datasets of labeled (image, identity) pairs to learn discriminative features. The key insight is that both stages – unsupervised discovery of facial structure and supervised association with identities – are necessary for robust recognition.

Exercise 1.1.2 (Supervised Learning Setup: Handwritten Digits)

How would you set up a supervised learning problem for a dataset of handwritten digit images (0–9)?

Solution.

- **Inputs** x: the pixel intensities of each image, e.g. $x \in \mathbb{R}^{784}$ for a 28×28 greyscale image flattened into a vector.
- **Outputs** y: the digit label, $y \in \{0, 1, \ldots, 9\}$ (10 classes).
- **Task**: *multi-class classification*.
- **Model types**: SoftMax regression (linear baseline), support vector machine with multi-class extension, or a convolutional neural network (state-of-the-art).
- **Loss**: categorical cross-entropy: $-\sum_k y_k \log \hat{y}_k$ (with one-hot encoding of y).

Exercise 1.2.1 (Train/Test Split)

You have 10,000 labeled samples. Using an 80/20 split: how many go into each set? What could happen if you used all data for training?

Solution.

- **Training set**: $10{,}000 \times 0.8 = 8{,}000$ samples.
- **Test set**: $10{,}000 \times 0.2 = 2{,}000$ samples.

If all data is used for training with no held-out set: the model may *overfit* — memorizing training labels rather than learning generalizable patterns. Without a test set, there is no reliable estimate of how the model will perform on unseen data. The reported training accuracy is an optimistic and misleading proxy for true predictive performance.

Exercise 1.2.2 (Standardization)

Compute the standardized version of $x = [2, 4, 6, 8, 10]$ and verify mean 0 and variance 1.

Solution. Sample mean: $\bar{x} = (2 + 4 + 6 + 8 + 10)/5 = 6$.

Sample variance: $s^2 = \frac{1}{5}\sum_{j=1}^{5}(x^{(j)} - 6)^2 = \frac{16+4+0+4+16}{5} = 8$, so $s = 2\sqrt{2}$.

Standardized values $z^{(j)} = (x^{(j)} - 6)/(2\sqrt{2})$:

$$z = \left[-\sqrt{2},\ -\frac{1}{\sqrt{2}},\ 0,\ \frac{1}{\sqrt{2}},\ \sqrt{2}\right].$$

Verification.

$$\bar{z} = \frac{1}{5}\left(-\sqrt{2} - \tfrac{1}{\sqrt{2}} + 0 + \tfrac{1}{\sqrt{2}} + \sqrt{2}\right) = 0.$$

$$\frac{1}{5}\sum_j (z^{(j)})^2 = \frac{1}{5}\left(2 + \tfrac{1}{2} + 0 + \tfrac{1}{2} + 2\right) = \frac{5}{5} = 1.$$

Exercise 1.2.3 (Loss Functions for Regression and Classification)

Give one example each of a regression loss and a classification loss, and explain what minimizing each achieves.

Solution.

1. **Regression — Mean Squared Error (MSE):** $\ell(y, \hat{y}) = (y - \hat{y})^2$. Minimizing MSE finds the parameter vector that minimizes the average squared deviation between predictions and true labels. Under a Gaussian noise model $y = f_\theta(x) + \epsilon$ with $\epsilon \sim \mathcal{N}(0, \sigma^2)$, minimizing MSE is equivalent to maximum likelihood estimation of θ.

2. **Classification — Binary cross-entropy:** $\ell(y, \hat{y}) = -y \log \hat{y} - (1 - y) \log(1 - \hat{y})$. Minimizing BCE finds parameters that maximize the likelihood of observing the binary labels under a Bernoulli model: $y^{(i)} \sim \text{Bernoulli}(\hat{y}^{(i)})$. It penalizes confident wrong predictions far more severely than uncertain correct ones, making it appropriate for calibrated probability estimation.

Exercise 1.3.1 (Binary Cross-Entropy: Limits and Domain)

(1) What is the binary cross-entropy loss when $y = 1$, $\hat{y} \to 1$? When $y = 1$, $\hat{y} \to 0$?

(2) Why is the loss undefined at $\hat{y} = 0$ or $\hat{y} = 1$, and why does this not cause problems when $\hat{y} = \sigma(f_\theta(x))$?

Solution.

(1) When $y = 1$: $\ell = -\log \hat{y}$.

- $\hat{y} \to 1$: $\ell = -\log(1) = 0$. A correct, confident prediction costs nothing.
- $\hat{y} \to 0$: $\ell = -\log(0) = +\infty$. A confident, wrong prediction is infinitely penalized.

(2) The loss contains $\log \hat{y}$ and $\log(1 - \hat{y})$, which are undefined at $\hat{y} = 0$ or $\hat{y} = 1$. However, $\hat{y} = \sigma(\theta^\top \tilde{x}) = 1/(1 + e^{-z})$ is strictly in $(0, 1)$ for any *finite* $z = \theta^\top \tilde{x}$: the sigmoid asymptotically approaches but never reaches 0 or 1. Hence the loss is always well-defined during training. (In floating-point implementations, numerical underflow is guarded against by clamping $\hat{y}$ to $[\varepsilon, 1 - \varepsilon]$ for a small $\varepsilon > 0$.)

Exercise 1.3.2 (Empirical Risk Minimization)

(1) For each loss in Table 1, write $C(\theta)$ explicitly as a sum over the n training examples.

(2) When f_θ is a linear model, which losses yield a convex objective in θ?

Solution.

(1) Empirical risks:

- **Squared loss**: $C(\theta) = \frac{1}{n}\sum_{i=1}^{n}\big(y^{(i)} - \theta^\top \tilde{x}^{(i)}\big)^2 = \frac{1}{n}\|y - X\theta\|^2$.
- **Binary cross-entropy**: $C(\theta) = -\frac{1}{n}\sum_{i=1}^{n}\Big[y^{(i)}\log\sigma\big(\theta^\top\tilde{x}^{(i)}\big) + (1-y^{(i)})\log\Big(1-\sigma\big(\theta^\top\tilde{x}^{(i)}\big)\Big)\Big]$.
- **Categorical cross-entropy**: $C(W) = -\frac{1}{n}\sum_{i=1}^{n}\sum_{k=1}^{K} y_k^{(i)}\log\hat{y}_k^{(i)}$, where $\hat{y}^{(i)} = \mathrm{softmax}(W\tilde{x}^{(i)})$.

(2) For a linear model $f_\theta(x) = \theta^\top\tilde{x}$, **all three** losses yield convex objectives in θ:

- Squared loss: Hessian $= \frac{2}{n}X^\top X \succeq 0$ (positive semi-definite).
- Binary cross-entropy: shown in Exercise 12 below, the Hessian is postive semi-definite.
- Categorical cross-entropy: the log-sum-exp function has a positive semi-definite Hessian, and composition with a linear map preserves convexity.

Exercise 1.4.1 (Normal Equations, Uniqueness, Pseudoinverse)

(1) Verify $\nabla_\theta\|y - X\theta\|^2 = -2X^\top y + 2X^\top X\theta$ and derive the normal equations.

(2) Under what condition on X does a unique solution exist?

(3) Show $\hat{\theta} = X^\dagger y$ satisfies the normal equations and has minimum Euclidean norm when solutions are not unique.

Solution.

(1) Expanding: $\|y - X\theta\|^2 = y^\top y - 2y^\top X\theta + \theta^\top X^\top X\theta$. Differentiating:

$$\nabla_\theta\|y - X\theta\|^2 = -2X^\top y + 2X^\top X\theta.$$

Setting to zero gives the *normal equations*: $X^\top X\,\theta = X^\top y$.

(2) A unique solution $\hat{\theta} = (X^\top X)^{-1}X^\top y$ exists if and only if $X^\top X$ is invertible, i.e., X has **full column rank** ($\mathrm{rank}(X) = p$, where p is the number of features). This holds when $n \geq p$ and no column of X is a linear combination of the others.

(3) Using the SVD $X = U\Delta V^\top$ and $X^\dagger = V\Delta^+U^\top$:

$$X^\top X\,X^\dagger y = V\Delta^\top U^\top\cdot U\Delta V^\top\cdot V\Delta^+U^\top y = V\Delta^\top\Delta\Delta^+U^\top y = V\Delta^\top U^\top y = X^\top y,$$

where we used $\Delta^\top \Delta \Delta^+ = \Delta^\top$ (since Δ^+ inverts the non-zero diagonals of $\Delta^\top \Delta$). So $X^\dagger y$ satisfies the normal equations.

For minimum norm, work directly with the SVD. Change variables by setting $\tilde{\theta} = V^\top \theta$ and $\tilde{u} = U^\top y$. Substituting $X = U\Delta V^\top$ into the normal equations $X^\top X\,\theta = X^\top y$ gives, componentwise,

$$\sigma_i^2\,\tilde{\theta}_i \;=\; \sigma_i\,\tilde{u}_i, \qquad i = 1, \ldots, p,$$

where $\sigma_1 \geq \cdots \geq \sigma_r > 0$ are the non-zero singular values. For $i \leq r$ this determines $\tilde{\theta}_i = \tilde{u}_i/\sigma_i$ uniquely; for $i > r$ the equation reads $0 \cdot \tilde{\theta}_i = 0$, so $\tilde{\theta}_i$ is free.

Now because V is orthogonal, $\|\theta\|^2 = \|V\tilde{\theta}\|^2 = \tilde{\theta}^\top V^\top V \tilde{\theta} = \|\tilde{\theta}\|^2$. Splitting the sum into determined and free components:

$$\|\theta\|^2 \;=\; \underbrace{\sum_{i=1}^{r} \big(\tilde{u}_i/\sigma_i\big)^2}_{\text{fixed by the equations}} \;+\; \underbrace{\sum_{i=r+1}^{p} \tilde{\theta}_i^2}_{\geq 0}.$$

The minimum is attained exactly when every free $\tilde{\theta}_i = 0$. That choice gives $\tilde{\theta} = \Delta^+ \tilde{u}$, so $\theta = V\Delta^+ U^\top y = X^\dagger y$. Hence $X^\dagger y$ is the unique minimum-norm solution.

Exercise 1.4.2 (Robustness of L_1 and Differentiability of Huber)

(1) Why is L_1 more robust to outliers than L_2? Give the gradient of each with respect to the residual r for large $|r|$.

(2) Show the Huber loss is differentiable everywhere by verifying continuity of the loss and its derivative at $|r| = \delta$.

Solution.

(1) For a residual $r = y^{(i)} - \hat{y}^{(i)}$:

- **L_2**: gradient $= 2r$, growing linearly in $|r|$ without bound. A single large outlier (large $|r|$) exerts an arbitrarily large gradient, pulling the fitted parameters strongly toward it and distorting the fit.
- **L_1**: gradient $= \text{sign}(r)$, bounded by ± 1 for all r. The influence of an outlier on the gradient is capped, so extreme residuals have limited effect on the estimator.

L_1 is therefore more robust: its gradient is bounded while L_2's is not.

(2) Huber loss at $|r| = \delta$ (from below and above):

	Loss value	**Derivative**
Quadratic region ($\lvert r\rvert \to \delta^-$):	$\frac{1}{2}\delta^2$	$r \to \pm\delta$
Linear region ($\lvert r\rvert \to \delta^+$):	$\delta \cdot \delta - \frac{1}{2}\delta^2 = \frac{1}{2}\delta^2$	$\pm\delta$

Both the loss value ($\frac{1}{2}\delta^2$) and the derivative ($\pm\delta$) match at the transition point. Hence the Huber loss is continuously differentiable everywhere, combining the smooth quadratic behaviour near zero with the bounded gradient of L_1 in the tails.

Exercise 1.4.3 (Design Matrix for Polynomial Regression)

(1) Write the design matrix X for the degree-2 polynomial model on n training points.

(2) Derive the normal equations and closed-form solution for $\hat{\theta} = (\hat{\theta}_0, \hat{\theta}_1, \hat{\theta}_2)^\top$.

Solution.

(1) The features for the degree-2 model $y = \theta_0 + \theta_1 x + \theta_2 x^2$ are $(1, x, x^2)$, giving design matrix:

$$X = \begin{bmatrix} 1 & x^{(1)} & (x^{(1)})^2 \\ \vdots & \vdots & \vdots \\ 1 & x^{(n)} & (x^{(n)})^2 \end{bmatrix} \in \mathbb{R}^{n\times 3}.$$

(2) The normal equations are $(X^\top X)\theta = X^\top y$, where:

$$X^\top X = \begin{bmatrix} n & \sum x^{(j)} & \sum (x^{(j)})^2 \\ \sum x^{(j)} & \sum (x^{(j)})^2 & \sum (x^{(j)})^3 \\ \sum (x^{(j)})^2 & \sum (x^{(j)})^3 & \sum (x^{(j)})^4 \end{bmatrix}, \quad X^\top y = \begin{bmatrix} \sum y^{(j)} \\ \sum x^{(j)} y^{(j)} \\ \sum (x^{(j)})^2 y^{(j)} \end{bmatrix}.$$

When X has full column rank (generically when $n \geq 3$ and the $x^{(j)}$ are not all equal), the unique closed-form solution is:

$$\hat{\theta} = (X^\top X)^{-1} X^\top y.$$

Exercise 1.4.4 (Categorical Features and Rank Deficiency)

(1) A dataset has one numerical feature and one categorical feature with $L = 4$ levels. Write the design matrix using $L - 1$ dummies (plus intercept) and state d.

(2) Explain why including all L dummies *and* an intercept makes X rank-deficient.

Solution.

(1) Using levels 2, 3, 4 as dummies (level 1 as reference), the design matrix is:

$$X = \begin{bmatrix} 1 & x^{(1)}_{\text{num}} & \mathbf{1}[z^{(1)} = 2] & \mathbf{1}[z^{(1)} = 3] & \mathbf{1}[z^{(1)} = 4] \\ \vdots & \vdots & \vdots & \vdots & \vdots \end{bmatrix} \in \mathbb{R}^{n\times 5}.$$

$d = 5$: intercept + 1 numerical feature + 3 dummy variables.

(2) If all $L = 4$ dummies are included alongside the intercept column (all ones), note that for every observation exactly one dummy is 1 and the rest are 0, so:

$$\mathbf{1}[z = 1] + \mathbf{1}[z = 2] + \mathbf{1}[z = 3] + \mathbf{1}[z = 4] = 1 \quad \text{for every row.}$$

The sum of the four dummy columns equals the intercept column. This is a perfect linear dependency: the five columns are linearly dependent, so $\operatorname{rank}(X) < d$, $X^\top X$ is singular, and the normal equations $X^\top X\theta = X^\top y$ have infinitely many solutions (no unique $\hat{\theta}$).

Exercise 1.5.1 (Logistic Loss: Reformulation and Convexity)

Using $\sigma'(z) = \sigma(z)(1 - \sigma(z))$:

(1) Show the binary cross-entropy for a single example equals $\log(1+e^z)-yz$ where $z = \theta^\top \tilde{x}$.

(2) Verify convexity by showing the Hessian with respect to θ is positive semidefinite.

Solution.

(1) With $z = \theta^\top \tilde{x}$, $\sigma(z) = 1/(1 + e^{-z})$:

$$\log \sigma(z) = -\log(1 + e^{-z}) = z - \log(1 + e^z), \quad \log(1 - \sigma(z)) = -\log(1 + e^z).$$

Substituting into $\ell = -y \log \sigma(z) - (1 - y) \log(1 - \sigma(z))$:

$$\ell = -y(z - \log(1 + e^z)) + (1 - y) \log(1 + e^z) = -yz + \log(1 + e^z) = \log(1 + e^z) - yz.$$

(2) First derivative with respect to θ:

$$\frac{\partial \ell}{\partial \theta} = \frac{e^z}{1 + e^z}\tilde{x} - y\tilde{x} = (\sigma(z) - y)\tilde{x}.$$

Second derivative (Hessian):

$$\nabla^2_\theta \ell = \frac{\partial}{\partial \theta}\big[(\sigma(z) - y)\tilde{x}\big] = \sigma'(z)\tilde{x}\tilde{x}^\top = \sigma(z)(1 - \sigma(z))\, \tilde{x}\tilde{x}^\top.$$

Since $\sigma(z)(1 - \sigma(z)) > 0$ for all z and $\tilde{x}\tilde{x}^\top \succeq 0$, the Hessian is a positive scalar multiple of the positive semidefinite matrix $\tilde{x}\tilde{x}^\top$, hence it is also positive semidefinite. Convexity of ℓ in θ follows; the empirical risk $C(\theta) = \frac{1}{n}\sum_i \ell_i$ is also convex (positive combination of convex functions).

Exercise 1.5.2 (Classification Metrics)

(1) Given $(\mathrm{TP}, \mathrm{FP}, \mathrm{TN}, \mathrm{FN}) = (36, 8, 61, 8)$, compute Accuracy, Precision, Recall, Specificity, and F_1.

(2) How would each metric change qualitatively if τ is decreased from 0.5 to 0.3?

(3) A test set has 950 negatives and 50 positives; a classifier always predicts negative. Compute its Accuracy, Recall, and Precision. Why is Accuracy misleading?

Solution.

(1) Total $= 36 + 8 + 61 + 8 = 113$.

$$\begin{aligned}
\text{Accuracy} &= \frac{36+61}{113} = \frac{97}{113} \approx 85.8\%,\\
\text{Precision} &= \frac{36}{36+8} = \frac{36}{44} = \frac{9}{11} \approx 81.8\%,\\
\text{Recall} &= \frac{36}{36+8} = \frac{36}{44} = \frac{9}{11} \approx 81.8\%,\\
\text{Specificity} &= \frac{61}{61+8} = \frac{61}{69} \approx 88.4\%,\\
F_1 &= \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} = \frac{9}{11} \approx 81.8\%.
\end{aligned}$$

(Since Precision = Recall here, F_1 equals both.)

(2) Decreasing τ from 0.5 to 0.3 classifies more samples as positive (lower threshold):

- **Recall** $\uparrow$: fewer positives are missed (fewer FN).
- **Precision** $\downarrow$: more negatives are misclassified as positive (more FP).
- **Specificity** $\downarrow$: more negatives incorrectly flagged.
- F_1: may rise or fall depending on which effect dominates.
- **Accuracy**: may increase or decrease depending on class balance.

(3) Always predicting negative: $\mathrm{TP} = 0$, $\mathrm{FP} = 0$, $\mathrm{TN} = 950$, $\mathrm{FN} = 50$.

$$\text{Accuracy} = \frac{0+950}{1000} = 95\%, \quad \text{Recall} = \frac{0}{0+50} = 0, \quad \text{Precision} = \frac{0}{0} = \text{undefined}.$$

Accuracy is misleading because the dataset is severely imbalanced: 95% of samples are negative, so always guessing negative achieves high accuracy while completely failing to detect any positive case. A model with zero recall is useless for any detection task (e.g., disease screening). Metrics such as Recall, Precision, and AUC better reflect performance on imbalanced datasets.

Exercise 1.6.1 (SoftMax: Dimensions, Reduction to Logistic, Argmax)

(1) For $K = 3$, $p = 2$: state dimensions of W, $\tilde{x}$, s, $\hat{y}$.

(2) Show that for $K = 2$, softmax regression reduces to logistic regression.

(3) Why does $\arg\max_k s_k = \arg\max_k \hat{y}_k$?

Solution.

(1) With $d = p + 1 = 3$ and $K = 3$:

$$W \in \mathbb{R}^{3\times 3}, \quad \tilde{x} \in \mathbb{R}^3, \quad s = W\tilde{x} \in \mathbb{R}^3, \quad \hat{y} = \text{softmax}(s) \in \mathbb{R}^3.$$

(2) For $K = 2$, let $\theta_k \in \mathbb{R}^{p+1}$ be the k-th row of W and $s_k = \theta_k^\top \tilde{x}$. Then:

$$\hat{y}_1 = \frac{e^{s_1}}{e^{s_1} + e^{s_2}} = \frac{1}{1 + e^{s_2 - s_1}} = \frac{1}{1 + e^{-(\theta_1 - \theta_2)^\top \tilde{x}}} = \sigma\big((\theta_1 - \theta_2)^\top \tilde{x}\big).$$

Setting $\theta = \theta_1 - \theta_2$: $\hat{y}_1 = \sigma(\theta^\top \tilde{x})$, which is exactly the logistic regression model. Only the difference $\theta_1 - \theta_2$ is identifiable, confirming one parameter vector is redundant for $K = 2$.

(3) The softmax is a strictly increasing function of each score (in the sense that $\hat{y}_k = e^{s_k} / \sum_j e^{s_j}$ is increasing in s_k and decreasing in s_j for $j \neq k$). More directly: since $e^{(\cdot)}$ is strictly monotone, $s_k > s_j \Leftrightarrow e^{s_k} > e^{s_j} \Leftrightarrow \hat{y}_k > \hat{y}_j$. Softmax preserves the *ranking* of the scores, so $\arg\max_k s_k = \arg\max_k \hat{y}_k$.

Exercise 1.7.1 (Role of c in Soft-Margin SVM)

Explain the role of c in the soft-margin SVM: what does the solution look like as $c \to \infty$ and as $c \to 0$?

Solution. The soft-margin SVM minimizes $\frac{1}{2}\|w\|^2 + c\sum_i \xi_i$ subject to margin constraints allowing slack $\xi_i \geq 0$.

- **As** $c \to \infty$: the penalty on margin violations $\sum_i \xi_i$ dominates. The optimizer is forced to set all $\xi_i \to 0$, enforcing the constraints $y^{(i)}(w^\top x^{(i)} + b) \geq 1$ exactly. The solution approaches the *hard-margin SVM*: the widest margin that commits no training errors. If the data are not linearly separable, $\|w\|$ must become very large (and the margin collapses) to satisfy the constraints.

- **As** $c \to 0$: the violation penalty disappears. The objective is dominated by $\frac{1}{2}\|w\|^2$, whose minimizer is $w = 0$ — a degenerate classifier that ignores all data. The margin becomes infinitely wide but the classifier has zero discriminative ability. Every training point is a margin violator ($\xi_i > 0$) and the classifier assigns the same score to all inputs.

The parameter c thus controls the **bias–variance trade-off**: large c fits training data tightly (low bias, high variance); small c regularizes aggressively (high bias, low variance). The optimal c is selected by cross-validation.

Exercise 1.8.1 (Bias–Variance Decomposition)

Prove the bias–variance decomposition by expanding $\mathbb{E}_{\mathcal{D}}[(y - \hat{f}_{\mathcal{D}}(x))^2]$ and using independence of ϵ from $\mathcal{D}$.

Solution. Write $y = f^*(x) + \epsilon$ with $\mathbb{E}[\epsilon] = 0$, $\mathbb{E}[\epsilon^2] = \sigma^2$, and $\epsilon \perp \mathcal{D}$. Let $\bar{f}(x) = \mathbb{E}_{\mathcal{D}}[\hat{f}_{\mathcal{D}}(x)]$. Decompose:

$$y - \hat{f}_{\mathcal{D}}(x) = \underbrace{(y - f^*(x))}_{\epsilon} + \underbrace{(f^*(x) - \bar{f}(x))}_{\text{Bias}(x)} + \underbrace{(\bar{f}(x) - \hat{f}_{\mathcal{D}}(x))}_{V}.$$

Expanding the square and taking $\mathbb{E}_{\mathcal{D}}[\cdot]$, all cross terms vanish:

- $\mathbb{E}[\epsilon \cdot \text{Bias}] = \text{Bias} \cdot \mathbb{E}[\epsilon] = 0$ (Bias is deterministic and $\mathbb{E}[\epsilon] = 0$).
- $\mathbb{E}[\epsilon \cdot V] = \mathbb{E}[\epsilon] \cdot \mathbb{E}[V] = 0$ (independence, $\mathbb{E}[\epsilon] = 0$).
- $\mathbb{E}[\text{Bias} \cdot V] = \text{Bias} \cdot \mathbb{E}[V] = \text{Bias} \cdot (\bar{f} - \bar{f}) = 0$.

Therefore:

$$\mathbb{E}_{\mathcal{D}}\Big[(y - \hat{f}_{\mathcal{D}}(x))^2\Big] = \mathbb{E}[\epsilon^2] + \text{Bias}^2(x) + \mathbb{E}[V^2] = \underbrace{\sigma^2}_{\text{Noise}} + \underbrace{(f^*(x) - \bar{f}(x))^2}_{\text{Bias}^2} + \underbrace{\mathbb{E}_{\mathcal{D}}\big[(\hat{f}_{\mathcal{D}}(x) - \bar{f}(x))^2\big]}_{\text{Variance}}. \quad \square$$

Exercise 1.8.2 (Ridge Regression: Derivation and Geometry)

(1) Derive the ridge solution $\hat{\theta}_\lambda = (X^\top X + n\lambda I)^{-1} X^\top y$ and show $X^\top X + n\lambda I$ is always invertible for $\lambda > 0$.

(2) Explain geometrically why L_1 promotes sparsity while L_2 does not.

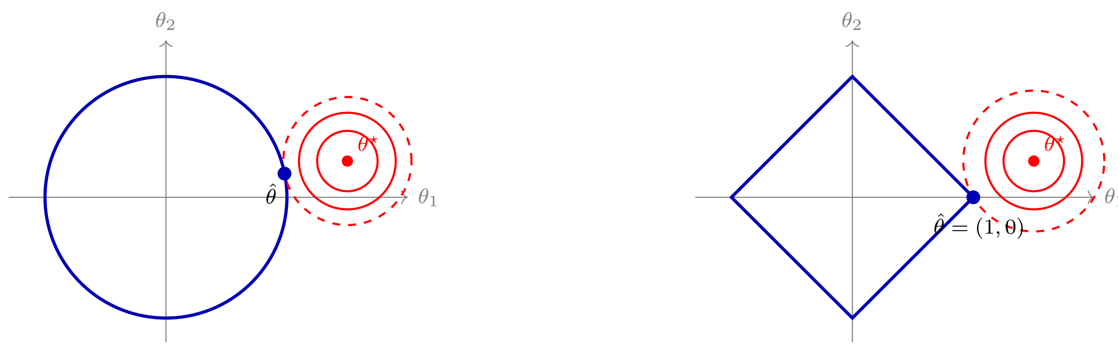
Solution.

(1) $C_\lambda(\theta) = \frac{1}{n}\|y - X\theta\|^2 + \lambda\|\theta\|^2$. Setting the gradient to zero:

$$\nabla_\theta C_\lambda = \frac{2}{n}(X^\top X\theta - X^\top y) + 2\lambda\theta = 0 \implies (X^\top X + n\lambda I)\theta = X^\top y \implies \hat{\theta}_\lambda = (X^\top X + n\lambda I)^{-1}X^\top y.$$

The eigenvalues of $X^\top X + n\lambda I$ are $\sigma_k^2 + n\lambda$ (where σ_k^2 are the eigenvalues of $X^\top X \geq 0$). For $\lambda > 0$: $\sigma_k^2 + n\lambda \geq n\lambda > 0$ for all k, so the matrix is positive definite and hence always invertible, even when $X^\top X$ is singular.

(2) The clearest way to see why is to picture the problem in two dimensions. Regularisation is equivalent to constraining θ to lie inside a region $R(\theta) \leq t$. The level sets of the unregularised loss $C(\theta)$ are concentric loops (circles, for the squared loss) around the unconstrained minimum $\theta^\star$. The constrained solution $\hat{\theta}$ is the point where the smallest such loop just touches the constraint region (Figure 1.1).



(a) L_2 constraint: round disc. The touching contour meets the boundary smoothly; both coordinates of $\hat{\theta}$ are nonzero.

(b) L_1 constraint: diamond. The corners poke outward toward $\theta^\star$, so the touching contour hits a corner. One coordinate of $\hat{\theta}$ is exactly zero.

Figure 1.1: Geometric view of regularisation in 2D. Blue: constraint region $R(\theta) \leq t$. Red: level sets of the unregularised loss $C(\theta)$, with the smallest touching contour drawn dashed. The constrained solution $\hat{\theta}$ sits at the touch point.

- With the $\mathbf{L}_2$ constraint $\theta_1^2 + \theta_2^2 \leq t$, the region is a round disc. The shrinking loop touches the disc at a smooth point on the circle, and both $\hat{\theta}_1$ and $\hat{\theta}_2$ are nonzero.
- With the $\mathbf{L}_1$ constraint $|\theta_1| + |\theta_2| \leq t$, the region is a diamond whose four sharp corners sit on the coordinate axes. Because the corners stick out toward $\theta^\star$, the shrinking loop is much more likely to hit a corner first than a flat edge. At a corner one of the coordinates is exactly zero, which is why L_1 produces a sparse solution.

In higher dimensions the L_1 ball has many vertices and lower-dimensional edges, all of which lie on subspaces where some coordinates are zero. Touching one of these faces is the typical outcome, so L_1 regularisation routinely sets many coefficients to zero exactly.

Exercise 1.9.1 (Gradients: Logistic and SoftMax)

(1) Derive $\nabla_\theta C(\theta) = \frac{1}{n} X^\top(\hat{y} - y)$ for logistic regression.

(2) Write the GD update for logistic regression explicitly.

(3) Write the GD update for the k-th row of W in SoftMax regression.

Solution.

(1) Let $\sigma_i = \sigma(\theta^\top \tilde{x}^{(i)})$. Differentiating $C(\theta) = -\frac{1}{n}\sum_i [y^{(i)} \log \sigma_i + (1 - y^{(i)}) \log(1 - \sigma_i)]$:

$$\frac{\partial C}{\partial \theta} = -\frac{1}{n} \sum_i \left[\frac{y^{(i)} \sigma_i'}{\sigma_i} - \frac{(1 - y^{(i)})\sigma_i'}{1 - \sigma_i} \right] \tilde{x}^{(i)}.$$

Using $\sigma_i' = \sigma_i(1 - \sigma_i)$:

$$= -\frac{1}{n} \sum_i \big[y^{(i)}(1 - \sigma_i) - (1 - y^{(i)})\sigma_i \big] \tilde{x}^{(i)} = -\frac{1}{n} \sum_i (y^{(i)} - \sigma_i) \tilde{x}^{(i)} = \frac{1}{n} X^\top(\hat{y} - y),$$

where $\hat{y}_i = \sigma_i$ and the matrix form stacks the terms.

(2) GD update for logistic regression (denoting $\hat{y}_i^{(t)} = \sigma(\theta^{(t)\top} \tilde{x}^{(i)})$):

$$\theta^{(t+1)} = \theta^{(t)} - \frac{\alpha}{n} X^\top \big(\hat{y}^{(t)} - y\big).$$

(3) For SoftMax regression, $\nabla_W C = \frac{1}{n} (\hat{Y} - Y)^\top X \in \mathbb{R}^{K \times d}$. The update for the k-th row W_k is:

$$W_k^{(t+1)} = W_k^{(t)} - \frac{\alpha}{n} \sum_{i=1}^{n} \big(\hat{y}_k^{(i)} - y_k^{(i)}\big) \tilde{x}^{(i)\top}.$$

Exercise 1.9.2 (L-Smoothness and Strong Convexity of Squared Loss)

(1) Verify that $C(\theta) = \frac{1}{n}\|y - X\theta\|^2$ is L-smooth with $L = 2\lambda_{\max}/n$.

(2) Show C is μ-strongly convex with $\mu = 2\lambda_{\min}/n$ when X has full column rank. What happens when $n < d$?

Solution.

(1) $\nabla C(\theta) = \frac{2}{n} X^\top (X\theta - y)$. Gradient difference:

$$\|\nabla C(\theta) - \nabla C(\theta')\| = \frac{2}{n}\|X^\top X(\theta - \theta')\| \leq \frac{2}{n}\|X^\top X\|_2 \|\theta - \theta'\| = \frac{2\lambda_{\max}}{n}\|\theta - \theta'\|,$$

using $\|X^\top X\|_2 = \lambda_{\max}(X^\top X)$. So C is L-smooth with $L = 2\lambda_{\max}/n$.

(2) The Hessian $\nabla^2 C = \frac{2}{n} X^\top X$. Strong convexity with constant μ requires $\nabla^2 C \succeq \mu I$, equivalently all eigenvalues of $\frac{2}{n} X^\top X \geq \mu$. The smallest eigenvalue of $\frac{2}{n} X^\top X$ is $\frac{2\lambda_{\min}}{n}$, giving $\mu = \frac{2\lambda_{\min}}{n}$ when $\lambda_{\min} > 0$, i.e., when X has full column rank.

When $n < d$: $X \in \mathbb{R}^{n\times d}$ has at most rank $n < d$, so $X^\top X$ has at least $d - n$ zero eigenvalues. Thus $\lambda_{\min} = 0$ and C is *not* strongly convex — only convex. The minimizer is not unique (infinitely many solutions exist).

Exercise 1.9.3 (GD Convergence for Linear Regression)

(1) Show $\|\theta^{(t)} - \theta^*\| \leq \rho(A)^t \|\theta^{(0)} - \theta^*\|$ for the GD iteration on linear regression.

(2) Express $\rho(A)$ in terms of eigenvalues of $X^\top X$ and α. Find the optimal α and resulting rate.

Solution.

(1) The GD iteration gives $\theta^{(t+1)} = A\theta^{(t)} + \frac{2\alpha}{n} X^\top y$ where $A = I - \frac{2\alpha}{n} X^\top X$. Any solution θ^* of the normal equations satisfies $A\theta^* + \frac{2\alpha}{n} X^\top y = \theta^*$ (by the same affine formula). Subtracting:

$$\theta^{(t+1)} - \theta^* = A(\theta^{(t)} - \theta^*).$$

Since A is symmetric, $\|A\|_2 = \rho(A)$, so:

$$\|\theta^{(t+1)} - \theta^*\| \leq \rho(A)\|\theta^{(t)} - \theta^*\|.$$

Applying this recursively: $\|\theta^{(t)} - \theta^*\| \leq \rho(A)^t \|\theta^{(0)} - \theta^*\|$. Convergence holds iff $\rho(A) < 1$, which requires $0 < \alpha < n/\lambda_{\max}$.

(2) The eigenvalues of A are $\{1 - 2\alpha\lambda_k/n\}$ for each eigenvalue λ_k of $X^\top X$. Hence:

$$\rho(A) = \max_k \left|1 - \frac{2\alpha\lambda_k}{n}\right| = \max\left(\left|1 - \frac{2\alpha\lambda_{\max}}{n}\right|, \left|1 - \frac{2\alpha\lambda_{\min}}{n}\right|\right).$$

The spectral radius is minimized by balancing the two extreme eigenvalues. Setting $\left|1 - \frac{2\alpha\lambda_{\max}}{n}\right| = 1 - \frac{2\alpha\lambda_{\min}}{n}$ (the outer eigenvalue negative, inner positive):

$$\frac{2\alpha\lambda_{\max}}{n} - 1 = 1 - \frac{2\alpha\lambda_{\min}}{n} \implies \alpha^* = \frac{n}{\lambda_{\max} + \lambda_{\min}}.$$

The resulting convergence rate:

$$\rho(A^*) = 1 - \frac{2\lambda_{\min}}{\lambda_{\max} + \lambda_{\min}} = \frac{\lambda_{\max} - \lambda_{\min}}{\lambda_{\max} + \lambda_{\min}} = \frac{\kappa - 1}{\kappa + 1},$$

where $\kappa = \lambda_{\max}/\lambda_{\min}$ is the condition number. Large κ gives $\rho \approx 1$ (slow convergence).

Exercise 1.9.4 (Correlated Features and Convergence as $r \to 1$)

(1) For two standardized features with correlation r, find the optimal α and convergence rate $\rho(A)$.

(2) What happens as $r \to 1$? Interpret geometrically.

Solution.

(1) The Gram matrix $Z^\top Z = n \begin{bmatrix} 1 & r \\ r & 1 \end{bmatrix}$ has eigenvalues $n(1 \pm |r|)$. With the standard least-squares objective $C(\theta) = \frac{1}{n}\|y - Z\theta\|^2$, the update matrix is $A = I - \frac{2\alpha}{n} Z^\top Z$, with eigenvalues $1 - 2\alpha(1 \pm |r|)$. The optimal step size minimises $\rho(A)$:

$$\alpha^* = \frac{n}{\lambda_{\max} + \lambda_{\min}} = \frac{n}{n(1 + |r|) + n(1 - |r|)} = \frac{1}{2}.$$

The resulting convergence rate:

$$\rho(A^*) = \frac{\lambda_{\max} - \lambda_{\min}}{\lambda_{\max} + \lambda_{\min}} = \frac{n(1 + |r|) - n(1 - |r|)}{n(1 + |r|) + n(1 - |r|)} = \frac{2|r|}{2} = |r|.$$

(2) As $r \to 1$: $\rho(A^*) = |r| \to 1$. The geometric convergence factor approaches 1, meaning the number of iterations to reach a given accuracy grows without bound: $\|\theta^{(t)} - \theta^*\| \le |r|^t \|\theta^{(0)} - \theta^*\| \to 0$ at rate $|r|^t$, which becomes arbitrarily slow.

Geometrically: when $r \to 1$ the two features become nearly collinear, the loss contours become extremely elongated ellipses (thin valleys aligned with the correlation direction). Gradient descent oscillates across the narrow valley with tiny steps in the useful direction, requiring many iterations to reach the minimum. Note that *standardization does not fix this*: z-scoring is an affine transformation and so leaves the Pearson correlation r unchanged. Standardization equalises feature *scales* (which itself improves conditioning when scales differ greatly), but once features are already standardised the conditioning is governed purely by $|r|$. Reducing $|r|$ requires an actual change of basis, e.g. PCA / decorrelation.

Exercise 1.9.5 (SGD: Unbiasedness and Epoch Cost)

(1) Show the mini-batch gradient is an unbiased estimator of $\nabla C(\theta)$.

(2) For $n = 10{,}000$ and $m = 100$: how many mini-batch steps constitute one epoch? Compare total cost of one epoch of SGD to one step of full GD.

Solution.

(1) Since $\mathcal{B}^{(t)}$ is sampled uniformly from $\{1, \ldots, n\}$ (each element of the batch is an independent uniform draw):

$$\begin{aligned}\mathbb{E}\left[\frac{1}{m}\sum_{i\in\mathcal{B}^{(t)}} \nabla\ell\big(y^{(i)}, f_\theta(x^{(i)})\big)\right] &= \frac{1}{m}\sum_{j=1}^{m} \mathbb{E}\Big[\nabla\ell(y^{(i_j)}, f_\theta(x^{(i_j)}))\Big] \\ &= \frac{1}{m}\cdot m \cdot \frac{1}{n}\sum_{i=1}^{n} \nabla\ell\big(y^{(i)}, f_\theta(x^{(i)})\big) \\ &= \nabla C(\theta).\end{aligned}$$

Each batch element is drawn uniformly from the training set, so its expected gradient equals the average over all n examples.

(2) With $n = 10{,}000$, $m = 100$: one epoch $= n/m = 100$ mini-batch steps.

- **SGD**: 100 steps $\times$ $O(m) = O(100)$ gradient evaluations per step $= O(10{,}000) = O(n)$ total.
- **Full GD**: 1 step $\times$ $O(n) = O(10{,}000)$ gradient evaluations $= O(n)$ total.

Both cost the same total number of per-sample gradient evaluations per epoch. The key difference is that SGD performs **100 parameter updates** per epoch while full GD performs only **1**. Each SGD update uses a noisy but unbiased gradient, enabling faster progress (more frequent corrections) at the cost of variance. For non-convex objectives (deep networks), the gradient noise also helps escape saddle points — a significant practical advantage.